

20. Bölüm

Zaman Uyumlu Programlama

Günümüz yazılım dünyasında uygulamalar yalnızca hesaplama yapmakla kalmaz; ağ üzerinden veri çeker, dosya sistemiyle etkileşime girer, veri tabanına sorgu gönderir. Bu işlemlerin tamamı **giriş-çıkış bağımlı (I/O-bound)** işlemlerdir ve donanımın yanıt vermesini bekler. Bu süre zarfında işlemciyi meşgul tutmak kaynakların israfıdır.

Eş zamansız programlama (asynchronous programming), bir işlemin tamamlanmasını beklerken iş parçacığını serbest bırakmayı ve bu süreçte başka işlerin yapılmasını sağlar. C#, bu paradigmayı `async/await` sözdizimi ile son derece okunabilir bir biçimde sunar.

Bölümün sonunda aşağıdaki soruları rahatça yanıtlayabileceksiniz:

- ◇ Eş zamanlı ve eş zamansız programlama arasındaki fark nedir?
- ◇ `Task` ve `Task<T>` nasıl kullanılır?
- ◇ `async/await` anahtar kelimeleri derleyici tarafından nasıl işlenir?
- ◇ İptal mekanizması nasıl kurulur?
- ◇ Eş zamansız kodda istisnalar nasıl ele alınır?
- ◇ `ValueTask<T>` ile `Task<T>` arasındaki fark nedir?
- ◇ Paralel programlama eş zamansız programlamadan nasıl ayrışır?

20.1 Eş Zamansız Programlamanın Temelleri

§20.1.1 Eş Zamanlı ve Eş Zamansız Arasındaki Fark

Eş zamanlı (synchronous) programlamada her işlem, bir önceki tamamlanmadan başlamaz. Bir dosya okunurken ya da ağdan veri çekilirken iş parçacığı (**thread**) bloke olur ve bekler. **Eş zamansız (asynchronous)** programlamada ise işlem başlatılır, iş parçacığı serbest kalır ve işlem tamamlandığında yürütme kaldığı yerden devam eder.

§20.1.2 G/Ç Bağımlı ve İşlemci Bağımlı İşlemler

Eş zamansız programlamanın hangi tür işlemlerde avantaj sağladığını anlamak için iki temel kategoriye ayırt etmek gerekir.

G/Ç bağımlı (I/O-bound) işlemler, işlemciyi değil donanımı meşgul eder: ağ isteği, dosya okuma/yazma, veri tabanı sorgusu. Bu işlemler için `async/await` idealdir.

İşlemci bağımlı (CPU-bound) işlemler ise hesaplama yoğunudur: görüntü işleme, şifreleme, fizik simülasyonu. Bu tür işlemler için `Task.Run()` ile ayrı bir iş parçacığına yük aktarılır.

Tür	Örnek	Önerilen Çözüm
G/Ç Bağımlı	HTTP isteği, dosya okuma	<code>await</code> ile eş zamansız
İşlemci Bağımlı	Şifreleme, görüntü işleme	<code>Task.Run()</code> ile paralel

Tablo 20.1: G/Ç Bağımlı ve İşlemci Bağımlı İşlemler

§20.1.3 .NET'te Eş Zamansız Programlamanın Tarihi

.NET'te eş zamansız programlama üç dönemde gelişmiştir:

- ◇ **APM (Asynchronous Programming Model)**: `BeginXxx/EndXxx` çift yöntemi biçimindeydi. Karmaşık ve hata eğilimliydi.
- ◇ **EAP (Event-based Asynchronous Pattern)**: `BackgroundWorker` ve olaylar kullanılıyordu. Okunabilirliği daha iyiydi, ancak yine de karmaşıktı.
- ◇ **TAP (Task-based Asynchronous Pattern)**: C#5.0 ile gelen `async/await` söz dizimi. Günümüzde standart olan yaklaşımdır.

20.2 Görev Tabanlı Programlama

`Task` sınıfı, `System.Threading.Tasks` isim uzayında tanımlıdır ve eş zamansız bir işlemi temsil eder. Tıpkı bir söz (promise) gibi düşünülebilir: “Bu işi başlatıyorum, bitince sana bildiririm.”

§20.2.1 Task Sınıfı

Değer döndürmeyen eş zamansız işlemler için `Task` kullanılır.

```
using System;
using System.Threading.Tasks;

// Değer döndürmeyen görev
Task görev = Task.Run(() =>
{
    Console.WriteLine("Görev arka planda çalışıyor.");
});

görev.Wait(); // Tamamlanmasını bekle (eş zamanlı bekleme)
Console.WriteLine("Görev tamamlandı.");
```

§20.2.2 Task<T> Sınıfı

Değer döndüren eş zamansız işlemler için `Task<T>` kullanılır. `T` dönüş tipidir.

```
using System;
using System.Threading.Tasks;

Task<int> hesaplaGörevi = Task.Run(() =>
{
    int toplam = 0;
    for (int i = 1; i <= 100; i++)
        toplam += i;
    return toplam;
});

int sonuç = hesaplaGörevi.Result; // Sonucu al (bloke eder!)
Console.WriteLine($"1'den 100'e toplam: {sonuç}");
```

İş Parçacıklarını Bloke Etme

`.Result` ve `.Wait()` çağrıları iş parçacığını bloke eder. Eş zamansız bağlamda `await` kullanmak tercih edilmelidir. `.Result` yalnızca eş zamanlı bağlamda veya test kodlarında kullanın.

§20.2.3 Görev Oluşturma Yöntemleri

```
// 1. Task.Run - İşlemci bağımlı işlemler için
Task<double> karekökGörevi = Task.Run(() => Math.Sqrt(144));

// 2. Task.FromResult - Hazır değeri görev olarak sarmalar
Task<string> hazırGörev = Task.FromResult("Merhaba");

// 3. Task.Delay - Belirtilen süre bekler (Thread.Sleep'in eş zamansız karşılığı)
Task bekleGörevi = Task.Delay(TimeSpan.FromSeconds(2));

// 4. Task.CompletedTask - Tamamlanmış boş görev
Task tamamlananGörev = Task.CompletedTask;
```

§20.2.4 Görev Durumları

Bir `Task` nesnesi yaşam döngüsü boyunca çeşitli durumlardan geçer.

Durum	Açıklama
Created	Oluşturuldu, henüz başlamadı
WaitingToRun	Zamanlayıcı kuyruğunda
Running	Yürütülüyor
RanToCompletion	Başarıyla tamamlandı
Faulted	İstisna fırlatıldı
Canceled	İptal edildi

Tablo 20.2: Task Durum Geçişleri

20.3 async ve await Anahtar Kelimeleri

`async` ve `await`, C# 5.0 ile dile eklenen ve TAP modelini son derece okunabilir kılan sözdizimi öğeleridir.

§20.3.1 Temel Kullanım

```
using System;
using System.Net.Http;
using System.Threading.Tasks;

class HaberServisi
{
    private readonly HttpClient _istemci = new();

    // async anahtar kelimesi yöntemi eş zamansız yapar
    // Task<string> dönüş tipi zorunludur (void yerine)
    public async Task<string> HaberleriGetirAsync()
    {
        // await, görev tamamlanana kadar yöntemi duraklatır
        // ancak iş parçacığını BLOKELEMEZ
        string içerik = await _istemci.GetStringAsync(
            "https://api.example.com/haberler");
        return içerik;
    }
}

class Program
{
    static async Task Main(string[] args)
    {
        HaberServisi servis = new();
        string haberler = await servis.HaberleriGetirAsync();
        Console.WriteLine($"Alınan veri boyutu: {haberler.Length} karakter");
    }
}
```

§20.3.2 Derleyici Dönüşümü

`async/await` bir söz dizimi şekeridir (*syntactic sugar*). Derleyici bu yapıyı bir durum makinesine (*state machine*) dönüştürür. Aşağıdaki iki kod işlevsel olarak eşdeğerdir:

```
// async/await ile - okunabilir biçim
public async Task<int> DeğerGetirAsync()
{
    int sonuç = await HesaplaAsync();
    return sonuç * 2;
}

// Derleyicinin ürettiğine yakın biçim - karmaşık
public Task<int> DeğerGetirAsync()
{
    return HesaplaAsync().ContinueWith(görev =>
    {
        int sonuç = görev.Result;
        return sonuç * 2;
    });
}
```

§20.3.3 async Yöntem İmzaları

`async` anahtar kelimesiyle işaretlenmiş yöntemler üç farklı dönüş tipi olabilir.

Kural: Olay işleyicileri dışında `async void` kullanmaktan kaçının. İstisnaları yakalamak imkânsız hale gelir.

Dönüş Tipi	Kullanım Amacı	Örnek
Task	Değer döndürmeyen eş zamansız	async Task YapAsync()
Task<T>	Değer döndüren eş zamansız	async Task<int> HesaplaAsync()
void	Yalnızca olay işleyicileri	async void Buton_Tıklandı(...)
ValueTask<T>	Performans kritik durum	async ValueTask<int> HızlıAsync()

Tablo 20.3: async Yöntem Dönüş Tipleri

§20.3.4 Birden Fazla Görevi Bekleme

Task.WhenAll ile Hepsini Bekle

Birden fazla görevi paralel başlatıp hepsinin tamamlanmasını bekler.

```
using System;
using System.Threading.Tasks;

class VeriAkümülatörü
{
    public async Task<int[]> TümVerileriGetirAsync()
    {
        // Üç sorgu paralel olarak başlatılır
        Task<int> sorgu1 = VeritabanıSorgusuAsync("SELECT COUNT(*) FROM Müşteriler");
        Task<int> sorgu2 = VeritabanıSorgusuAsync("SELECT COUNT(*) FROM Siparişler");
        Task<int> sorgu3 = VeritabanıSorgusuAsync("SELECT COUNT(*) FROM Ürünler");

        // Üçü de bitene kadar bekle
        int[] sonuçlar = await Task.WhenAll(sorgu1, sorgu2, sorgu3);

        Console.WriteLine($"Müşteriler: {sonuçlar[0]}");
        Console.WriteLine($"Siparişler: {sonuçlar[1]}");
        Console.WriteLine($"Ürünler: {sonuçlar[2]}");

        return sonuçlar;
    }

    private async Task<int> VeritabanıSorgusuAsync(string sorgu)
    {
        await Task.Delay(500); // Veritabanı gecikmesini simüle et
        return new Random().Next(100, 1000);
    }
}
```

Task.WhenAny ile İlk Biteni Yakala

Birden fazla görev arasından ilk tamamlananı yakalar. Zaman aşımı (timeout) senaryolarında kullanışlıdır.

```
using System;
using System.Threading.Tasks;

class ZamanAşımıÖrneği
{
    public async Task<string> ZamanAşımıİleGetirAsync()
    {
        Task<string> veriGetirGörevi = VeriyiGetirAsync();
        Task zamanAşımıGörevi = Task.Delay(TimeSpan.FromSeconds(3));

        // Hangisi önce biterse
        Task tamamlanan = await Task.WhenAny(veriGetirGörevi, zamanAşımıGörevi);

        if (tamamlanan == veriGetirGörevi)
            return await veriGetirGörevi; // Veri zamanında geldi
        else
            throw new TimeoutException("Veri 3 saniyede gelmedi.");
    }
}
```

```

private async Task<string> VeriyiGetirAsync()
{
    await Task.Delay(2000); // 2 saniyelik ağ gecikmesi
    return "Sunucudan gelen veri";
}

```

§20.3.5 ConfigureAwait(false) ile Bağlam Yönetimi

`await` varsayılan olarak mevcut eşitleme bağlamına (`synchronization context`) geri döner. Kütüphane kodunda bu gereksiz yük oluşturur.

```

// Kütüphane kodu – UI bağlamına dönmek gerekmez
public async Task<string> KütüphaneYöntemAsync()
{
    // ConfigureAwait(false) ile UI thread'e geri dönme
    string veri = await HttpİsteğiAsync().ConfigureAwait(false);
    return veri.ToUpper();
}

// Uygulama kodu – UI güncellemesi gerektiğinde ConfigureAwait kullanma
private async void YenileButon_Tıklandı(object sender, EventArgs e)
{
    // UI thread'e geri dönmesi gerekir; ConfigureAwait(false) YOK
    string sonuç = await VeriyiGetirAsync();
    etiket.Text = sonuç; // UI güncellemesi
}

```

Kural: Kütüphane ve altyapı kodlarında `ConfigureAwait(false)` kullanın. Uygulama katmanı ve UI kodlarında kullanmayın.

20.4 İptal Belirteci

Uzun süren eş zamansız işlemlerin kullanıcı isteğiyle veya zaman aşımıyla iptal edilebilmesi gerekir. .NET bu amaç için `CancellationToken` ve `CancellationTokenSource` çiftini sunar.

§20.4.1 Temel İptal Mekanizması

```

using System;
using System.Threading;
using System.Threading.Tasks;

class Dosyaİşlemcisi
{
    // Yöntem, dışarıdan iptal belirteci kabul eder
    public async Task BüyükDosyayıİşleAsync(
        string dosyaYolu,
        CancellationToken iptalBelirteci = default)
    {
        Console.WriteLine("Dosya işleme başladı...");

        for (int i = 0; i < 100; i++)
        {
            // Her döngüde iptal istendi mi kontrol et
            iptalBelirteci.ThrowIfCancellationRequested();

            await Task.Delay(100, iptalBelirteci); // İptal belirtecini Delay'e de ver
            Console.WriteLine($"İşlenen satır: {i + 1}/100");
        }

        Console.WriteLine("Dosya işleme tamamlandı.");
    }
}

```

```

class Program
{
    static async Task Main(string[] args)
    {
        using CancellationTokenSource kaynak = new();
        Dosyaİşlemcisi işlemci = new();

        // 2 saniye sonra iptal et
        kaynak.CancelAfter(TimeSpan.FromSeconds(2));

        try
        {
            await işlemci.BüyükDosyayıİşleAsync("veri.txt", kaynak.Token);
        }
        catch (OperationCanceledException)
        {
            Console.WriteLine("İşlem kullanıcı tarafından iptal edildi.");
        }
    }
}

/*Program Çıktısı:
Dosya işleme başladı...
İşlenen satır: 1/100
İşlenen satır: 2/100
...
İşlenen satır: 19/100
İşlem kullanıcı tarafından iptal edildi.
*/

```

§20.4.2 Manuel İptal

```

using System;
using System.Threading;
using System.Threading.Tasks;

class Program
{
    static async Task Main(string[] args)
    {
        using CancellationTokenSource kaynak = new();

        // Kullanıcı tuşa basınca iptal et
        Console.WriteLine("Herhangi bir tuşa basınca işlem iptal edilir.");

        Task işlemGörevi = UzunİşlemAsync(kaynak.Token);

        // Arka planda tuş bekle
        Task tuşGörevi = Task.Run(() =>
        {
            Console.ReadKey(intercept: true);
            kaynak.Cancel(); // İptali tetikle
        });

        try
        {
            await Task.WhenAny(işlemGörevi, tuşGörevi);
            await işlemGörevi; // İstisna varsa yeniden fırlat
        }
        catch (OperationCanceledException)
        {
            Console.WriteLine("\nİşlem iptal edildi!");
        }
    }
}

```

```

static async Task UzunİşlemAsync(CancellationToken iptal)
{
    for (int i = 0; i < 50; i++)
    {
        iptal.ThrowIfCancellationRequested();
        Console.WriteLine($"{i + 1}/50 tamamlandı...");
        await Task.Delay(300, iptal);
    }
    Console.WriteLine("\nİşlem başarıyla tamamlandı.");
}

```

§20.4.3 Bağlantılı İptal Kaynakları

Birden fazla iptal kaynağını birleştirerek karma koşullarla iptal sağlanabilir.

```

using System;
using System.Threading;
using System.Threading.Tasks;

// Kullanıcı isteği veya zaman aşımı – hangisi önce gelirse iptal et
using CancellationTokenSource kullanıcıKaynağı = new();
using CancellationTokenSource zamanAşımıKaynağı = new(TimeSpan.FromSeconds(5));

// İki kaynağı birleştir
using CancellationTokenSource birleşik =
    CancellationTokenSource.CreateLinkedTokenSource(
        kullanıcıKaynağı.Token,
        zamanAşımıKaynağı.Token);

try
{
    await VeriGetirAsync(birleşik.Token);
}
catch (OperationCanceledException)
{
    if (zamanAşımıKaynağı.IsCancellationRequested)
        Console.WriteLine("Zaman aşımı nedeniyle iptal edildi.");
    else
        Console.WriteLine("Kullanıcı tarafından iptal edildi.");
}

```

20.5 İstisna Yönetimi Eş Zamansız Bağlamda

Eş zamansız programlamada istisna yönetimi, eş zamanlı koddan birkaç önemli farkla ayrışır.

§20.5.1 Temel İstisna Yakalama

```

using System;
using System.Net.Http;
using System.Threading.Tasks;

class HaberServisi
{
    private readonly HttpClient _istemci = new();

    public async Task<string> GüvenliGetirAsync(string url)
    {
        try
        {
            string veri = await _istemci.GetStringAsync(url);
            return veri;
        }
        catch (HttpRequestException hre)
        {
        }
    }
}

```

```

    {
        Console.WriteLine($"Ağ hatası: {hre.Message}");
        return string.Empty;
    }
    catch (TaskCanceledException)
    {
        Console.WriteLine("İstek zaman aşımına uğradı.");
        return string.Empty;
    }
}
}

```

§20.5.2 Birden Fazla Hatayı Toplama

`Task.WhenAll` ile birden fazla görev paralel çalıştırıldığında, birden fazla istisna oluşabilir. Bu istisnalar `AggregateException` içinde toplanır.

```

using System;
using System.Threading.Tasks;

class Program
{
    static async Task Main(string[] args)
    {
        Task görev1 = HatalıİşlemAsync("Görev 1");
        Task görev2 = HatalıİşlemAsync("Görev 2");
        Task görev3 = BaşarılıİşlemAsync();

        try
        {
            await Task.WhenAll(görev1, görev2, görev3);
        }
        catch (Exception)
        {
            // Her görevin durumunu ayrı ayrı incele
            foreach (Task görev in new[] { görev1, görev2, görev3 })
            {
                if (görev.IsFaulted)
                {
                    // AggregateException içindeki gerçek hataya eriş
                    Exception gerçekHata = görev.Exception!.InnerException!;
                    Console.WriteLine($"Hata: {gerçekHata.Message}");
                }
            }
        }
    }

    static async Task HatalıİşlemAsync(string ad)
    {
        await Task.Delay(100);
        throw new InvalidOperationException($"{ad} başarısız oldu.");
    }

    static async Task BaşarılıİşlemAsync()
    {
        await Task.Delay(50);
        Console.WriteLine("Görev 3 başarıyla tamamlandı.");
    }
}

/*Program Çıktısı:
Görev 3 başarıyla tamamlandı.
Hata: Görev 1 başarısız oldu.
Hata: Görev 2 başarısız oldu.

```


*/

§20.5.3 async void ve İstisna Tuzağı

`async void` yöntemlerinde fırlatılan istisnalar yakalanamaz ve uygulamayı çökertebilir.

```
// YANLIŞ – istisna yakalanamaz
async void YanlışKullanım()
{
    await Task.Delay(100);
    throw new Exception("Bu istisna kaybolur!"); // Uygulama çöker
}

// DOĞRU – yalnızca olay işleyicilerinde void kullan, içinde yakala
async void Buton_Tıklandı(object sender, EventArgs e)
{
    try
    {
        await EşZamansızİşlemAsync();
    }
    catch (Exception hata)
    {
        MesajGöster(hata.Message); // Hatayı kullanıcıya bildir
    }
}
```

20.6 ValueTask<T> ile Performans Optimizasyonu

`Task<T>` her çağrıda öbek belleğinde (heap) yeni bir nesne oluşturur. Yüksek frekanslı yöntemlerde bu durum çöp toplayıcıya (GC) baskı yapar. `ValueTask<T>`, sonucun eş zamanlı olarak hazır olduğu durumlarda öbek bellek tahsisini önler.

§20.6.1 Ne Zaman ValueTask<T> Kullanılmalı?

- ◊ Yöntem çoğunlukla eş zamanlı (kısık sürede) tamamlanıyorsa
- ◊ Saniyede binlerce kez çağrılan sıcak yollarda (hot path)
- ◊ Bellek tahsisi darboğaz oluşturuyorsa

```
using System;
using System.Threading.Tasks;
using System.Collections.Generic;

class ÖnbellekServisi
{
    private readonly Dictionary<string, string> _önbellek = new();

    // ValueTask<T> – çoğunlukla önbellekten (eş zamanlı) döner
    public ValueTask<string> DeğerGetirAsync(string anahtar)
    {
        // Önbellekte varsa eş zamanlı dön – tahsis yok!
        if (_önbellek.TryGetValue(anahtar, out string? değer))
            return new ValueTask<string>(değer);

        // Yoksa ağdan getir – gerçek eş zamansız işlem
        return new ValueTask<string>(AğdanGetirAsync(anahtar));
    }

    private async Task<string> AğdanGetirAsync(string anahtar)
    {
        await Task.Delay(500); // Ağ gecikmesi
        string değer = $"Değer_{anahtar}";
        _önbellek[anahtar] = değer;
        return değer;
    }
}
```

```

class Program
{
    static async Task Main(string[] args)
    {
        ÖnbellekServisi servis = new();

        // İlk çağrı - ağdan gelir (eş zamansız)
        string v1 = await servis.DeğerGetirAsync("kullanıcı:1");
        Console.WriteLine($"İlk çağrı: {v1}");

        // İkinci çağrı - önbellekten gelir (eş zamanlı, tahsis yok)
        string v2 = await servis.DeğerGetirAsync("kullanıcı:1");
        Console.WriteLine($"İkinci çağrı: {v2}");
    }
}

```

Özellik	Task<T>	ValueTask<T>
Bellek tahsisi	Her zaman öbek	Eş zamanlı durumda yok
Kullanım kolaylığı	Yüksek	Biraz daha karmaşık
Birden fazla await	Desteklenir	Desteklenmez
Ne zaman tercih edilmeli	Genel kullanım	Sıcak yollar, önbellek

Tablo 20.4: Task<T> ve ValueTask<T> Karşılaştırması

20.7 Paralel Programlama

Eş zamansız programlama G/Ç bağımlı işlemler için tasarlanmışken, **paralel programlama** işlemci bağımlı işlemleri çok çekirdekli işlemcilerde aynı anda yürütmek için tasarlanmıştır.

§20.7.1 Eş Zamansız ve Paralel Arasındaki Fark

Özellik	Eş Zamansız	Paralel
Hedef	G/Ç bağımlı	İşlemci bağımlı
İş parçacığı	Az (genelde 1)	Çok (çekirdek sayısı)
Mekanizma	async/await	Parallel, PLINQ
Örnek	HTTP isteği, DB sorgusu	Görüntü işleme, şifreleme

Tablo 20.5: Eş Zamansız ve Paralel Programlama Karşılaştırması

§20.7.2 Parallel.For ve Parallel.ForEach

```

using System;
using System.Threading.Tasks;

class Matrisİşlemcisi
{
    // Eş zamanlı versiyon - yavaş
    public static void EşZamanlıTopla(int[] dizi)
    {
        for (int i = 0; i < dizi.Length; i++)
            dizi[i] *= 2;
    }

    // Paralel versiyon - çok çekirdekli işlemcilerde hızlı
    public static void ParalelTopla(int[] dizi)
    {
        Parallel.For(0, dizi.Length, i =>
        {
            dizi[i] *= 2;
        });
    }
}

```

```

class Program
{
    static void Main(string[] args)
    {
        int[] veri = new int[10_000_000];
        for (int i = 0; i < veri.Length; i++)
            veri[i] = i;

        // Paralel.For ile işle
        Parallel.For(0, veri.Length, i =>
        {
            veri[i] = veri[i] * veri[i]; // Her elemanın karesini al
        });

        Console.WriteLine($"İlk 5 eleman: {veri[0]}, {veri[1]}, " +
            $"{veri[2]}, {veri[3]}, {veri[4]}");
    }
}
/*Program Çıktısı:
İlk 5 eleman: 0, 1, 4, 9, 16
*/
\end{csharp}

\subsubsection{Paralel Seçenekleri – \texttt{ParallelOptions}}

\begin{minted}{csharp}
using System;
using System.Threading;
using System.Threading.Tasks;

using CancellationTokenSource kaynak = new();

ParallelOptions seçenekler = new()
{
    MaxDegreeOfParallelism = Environment.ProcessorCount, // Çekirdek sayısı kadar
    CancellationToken = kaynak.Token
};

try
{
    Parallel.For(0, 1000, seçenekler, i =>
    {
        // Her yineleme için iptal kontrolü otomatik yapılır
        Console.Write($"{i} ");
    });
}
catch (OperationCanceledException)
{
    Console.WriteLine("\nParalel işlem iptal edildi.");
}

```

§20.7.3 PLINQ

Paralel LINQ (Kısaca PLINQ), LINQ sorgularını paralel olarak çalıştırır. Yalnızca `.AsParallel()` çağrısı eklemek yeterlidir.

```

using System;
using System.Linq;
using System.Collections.Generic;

class PLINQÖrneği
{

```

```

static void Main(string[] args)
{
    List<int> sayılar = Enumerable.Range(1, 10_000_000).ToList();

    // Eş zamanlı LINQ
    var eşZamanlıSonuç = sayılar
        .Where(s => s % 2 == 0)
        .Select(s => s * s)
        .Sum();

    // Paralel LINQ – çok çekirdekli işlemcilerde otomatik hızlanır
    var paralelSonuç = sayılar
        .AsParallel()
        .Where(s => s % 2 == 0)
        .Select(s => s * s)
        .Sum();

    Console.WriteLine($"Eş zamanlı: {eşZamanlıSonuç}");
    Console.WriteLine($"Paralel: {paralelSonuç}");
}

```

PLINQ Sıralı Çıktı

Paralel sorgunun sonuçları sırasız gelebilir. Sıra önemliyse `AsOrdered()` kullanın.

```

// Sırasız (daha hızlı)
var sonuç = Enumerable.Range(1, 100)
    .AsParallel()
    .Select(i => i * i)
    .ToList();

// Sıralı (kaynak sırasını korur, biraz daha yavaş)
var sıralıSonuç = Enumerable.Range(1, 100)
    .AsParallel()
    .AsOrdered()
    .Select(i => i * i)
    .ToList();

```

§20.7.4 İş Parçacığı Güvenliği ve Paylaşılan Veri

Paralel programlamada birden fazla iş parçacığı aynı veriye eriştiğinde **yarış koşulu** (race condition) oluşabilir.

```

using System;
using System.Threading;
using System.Threading.Tasks;

class YarışKoşuluÖrneği
{
    static int sayaç = 0;

    static void Main(string[] args)
    {
        // YANLIŞ – yarış koşulu var
        Parallel.For(0, 10000, i =>
        {
            sayaç++; // Atom değil! Veri kaybı olabilir
        });
        Console.WriteLine($"Beklenen: 10000, Gerçek: {sayaç}");

        // DOĞRU – Interlocked ile atomik artırım
        sayaç = 0;
        Parallel.For(0, 10000, i =>
        {

```

```

        Interlocked.Increment(ref sayaç); // Atom işlem, güvenli
    });
    Console.WriteLine($"Interlocked ile: {sayaç}");
}
}
/*Program Çıktısı (yaklaşık):
Beklenen: 10000, Gerçek: 9347    <- hatalı
Interlocked ile: 10000          <- doğru
*/

```

lock ile Kritik Bölge Koruma

```

using System;
using System.Threading;
using System.Threading.Tasks;

class BankaHesabı
{
    private decimal _bakiye;
    private readonly object _kilit = new(); // Kilit nesnesi

    public BankaHesabı(decimal başlangıçBakiye)
    {
        _bakiye = başlangıçBakiye;
    }

    public void ParaYatır(decimal miktar)
    {
        lock (_kilit) // Kritik bölgeye yalnızca bir thread girer
        {
            _bakiye += miktar;
        }
    }

    public bool ParaÇek(decimal miktar)
    {
        lock (_kilit)
        {
            if (_bakiye >= miktar)
            {
                _bakiye -= miktar;
                return true;
            }
            return false;
        }
    }

    public decimal Bakiye
    {
        get { lock (_kilit) { return _bakiye; } }
    }
}

```

20.8 Düşün ve Çöz

- ◊ **Düşün:** `async void` neden tehlikelidir? Olay işleyicileri dışında neden kullanılmamalıdır? `async void` içinde istisna fırlatıldığında ne olur? Bunu test etmek için bir örnek yazınız.
- ◊ **Düşün:** `Task.WhenAll` ve `Task.WhenAny` arasındaki fark nedir? Bir hava durumu uygulaması birden fazla kaynaktan veri çekiyorsa hangisini kullanmak daha mantıklıdır? Nedenini açıklayınız.
- ◊ **Düşün:** Aşağıdaki kod kilitlenmeye (deadlock) yol açar mı? Nedenini açıklayınız:

```

static async Task<int> DeğerGetirAsync()
{
    await Task.Delay(1000);
}

```

```

        return 42;
    }

    static void Main()
    {
        int sonuç = DeğerGetirAsync().Result; // Burada ne olur?
        Console.WriteLine(sonuç);
    }

```

- ◇ **Çöz:** Üç farklı sunucudan hava durumu verisi çeken bir uygulama yazınız. Tüm istekleri paralel gönderin. İlk 5 saniyede yanıt gelmeyen sunucular iptal edilsin. Yanıt gelen sunucuların verilerini ekrana yazdırın.
- ◇ **Çöz:** Büyük bir tam sayı listesinin asal sayılarını bulan bir program yazınız. Önce eş zamanlı LINQ ile çalışan versiyonu yazın. Ardından PLINQ versiyonunu yazın. **Stopwatch** kullanarak her iki versiyonun çalışma sürelerini karşılaştırın.
- ◇ **Düşün:** **CancellationToken** bir yöntem zincirinden geçirilirken ($A \rightarrow B \rightarrow C$) en iyi uygulama nedir? Belirtecisi her yöntem parametre olarak geçirmek yerine başka bir yöntem var mıdır? Avantaj ve dezavantajlarını tartışınız.
- ◇ **Çöz:** Bir banka uygulaması düşününüz. Aynı hesaba birden fazla iş parçacığı para yatırıp çekiyor. **lock** ve **SemaphoreSlim** kullanarak iki farklı güvenli versiyon yazın. Her iki yaklaşımın ne zaman daha uygun olduğunu açıklayın.